

ESP32 Win Monitor

Part 1: Software Guide

IMPORTANT: As of current, this is only tested/compatible with windows 11 - hence “Win” Monitor.
It *may* in future have extended functionality for other OS.

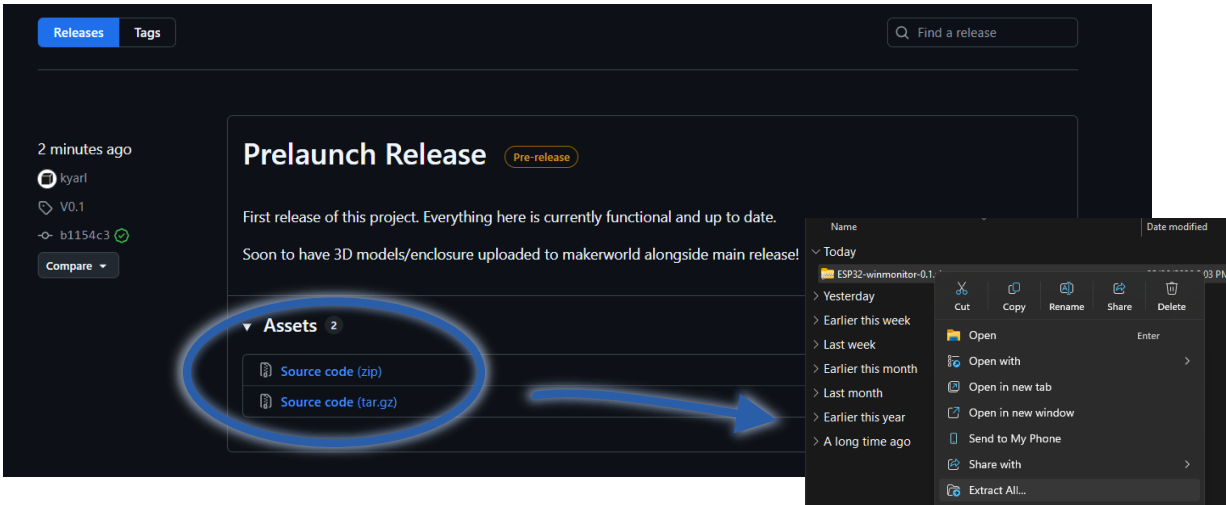
You should be relatively comfortable with Arduinio IDE and Python for this project.
If you get stuck, ask google first.

Most of the initial codebase is pretty crap, and there is lots of room for improving performance/making proper use of the S3. So, feel free/it is encouraged for you to contribute, just make sure you are aware of the project’s license.

Downloading Everything

All the software for this is hosted on GitHub:
<https://github.com/kyarl/ESP32-winmonitor>
(CC BY-NC-SA 4.0)

Download the latest version by visiting the [Releases Page](#):



Download & extract the zip file which contains everything you need.

You should put the extracted files/folders somewhere they won’t be deleted, i.e. your documents folder

It will contain the following:

Name	Date modified	Type
Today		
Python App	25/06/2026 5:04 PM	File folder
Arduino Sketches	25/06/2026 5:04 PM	File folder

As the folder names suggest. One has the python app, the other has all the Arduino sketches for programming your ESP32 board.

Uploading to ESP32 – Arduino IDE

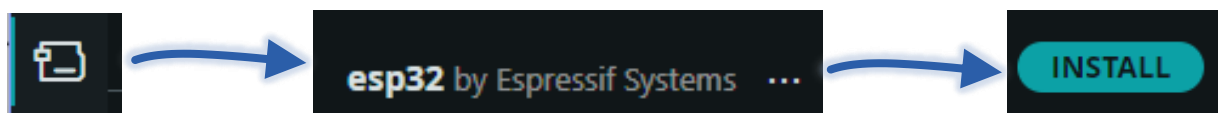
You don't need to have everything wired up yet, but it might be worthwhile to test functionality before you assemble the case.

Wiring diagrams & details are on [GitHub](#) / [MakerWorld](#).

Wiring is DIFFERENT depending on ESP32 variants.

Setup Arduino IDE for ESP32 Boards

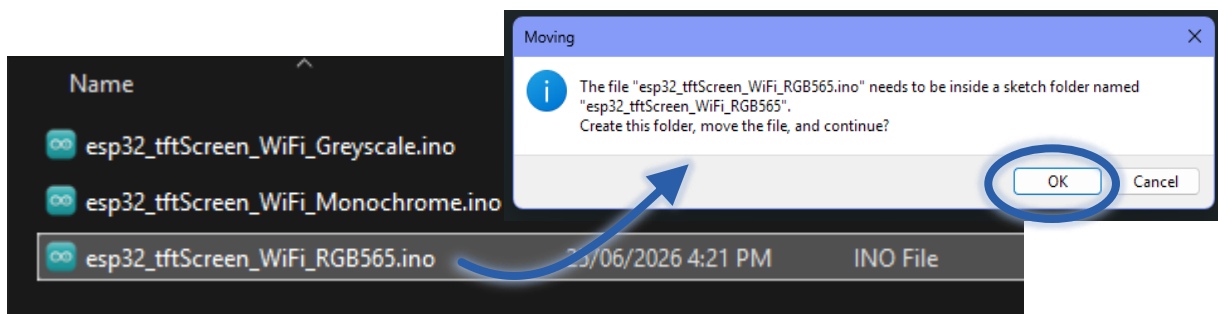
If you haven't already, you must enable ESP32 support.



Programming The Board

There are 3 Arduino .ino files to choose from:

I recommend starting with the RGB565 code, python is already set up for this.



The important things to change/check here are:

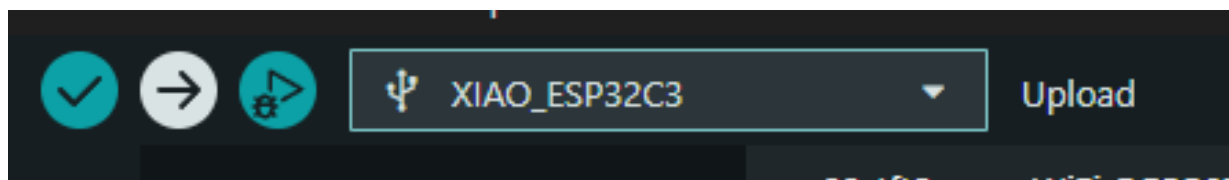
- Wi-Fi network name & pass
- Pinout (XIAO vs Supermini)

```
45 // WIFI Credentials
46 const char* ssid = "your wifi name";
47 const char* password = "password";
48
```

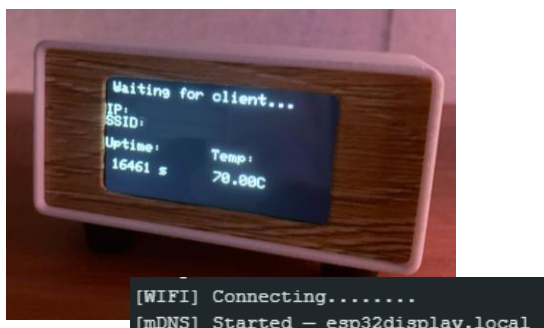
```
8 // Pinouts:
9 // - Note, you dont need to define the pins for SPI, unless you use non-default ones.
10
11 // GENERIC ESP32 S3 Supermini (NOT C3).
12 // Check YOUR board's silkscreen; SuperMini variants differ.
13 //
14 // BLK (LED) = GPIO 3
15 // CS = GPIO 10
16 // DC = GPIO 4
17 // RST = GPIO 5
18 // SDA/MOSI = GPIO 11 (SPI MOSI default)
19 // SCL/SCK = GPIO 12 (SPI SCK default)
20
21 // XIAO ESP32C3:
22 // mosi / sda connected to D10 (GPIO10 default xiao)
23 // miso NC (not connected)
24 // scl / sck connected to D8 (GPIO8 default xiao)
25 #define TFT_CS 6 // GPIO6 (D4)
26 #define TFT_DC 2 // GPIO2 (D0)
27 #define TFT_RST 3 // GPIO3 (D1)
28 #define TFT_BLK 7 // GPIO7 (D5)
```

If you have different hardware (screen size, etc) – you will have to change more.

Upload to ESP32



If you had things wired up correctly for testing, then you should see the following screen displaying some stats once it connects to Wi-Fi.



Streaming Windows Monitor – Python App

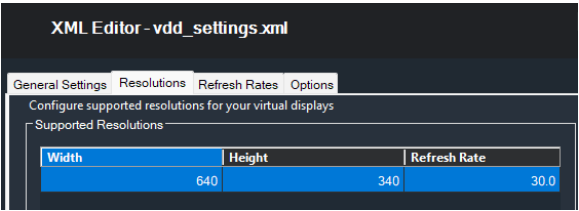
Now that the ESP32 is programmed, need to actually start sending frames/images for it to display.

VDD (Virtual Display Driver) - [THIS] Tutorial May Help

Allows ESP32 Win Monitor to **act as its own screen**, rather than just a teeny tiny copy of your main screen.

Main settings you want to edit in **vdd_settings.xml**∗:

- Resolution: 640 x 340
- Refresh Rate: 30Hz



- ∗ Technically could be any multiple of 320x170 (SPI Screen).
- ∗ And I would be surprised if can get more than 30fps on this.

This is a whole other project, made by someone else, completely unrelated to me. It is **NOT strictly required** for this mini monitor to work, so I will not be providing excessive details on setting this up.

There may be alternatives to VDD. There also may be a newer version of VDD.

Editing the Python App

If you are not using VDD.

Adjust **TARGET_MONITOR_SIZE**.

```
### VERY VERY IMPORTANT VARIABLE
TARGET_MONITOR_SIZE = (640, 340)    ## Format is (width, height) of the T
# IF THERE IS NO MONITOR FOUND OF THIS SIZE, THE PROGRAM WILL WAIT UNTIL Q
# Use VDD if you want a simulated monitor to be dedicated to this esp32
# (eg if you have a 640x340 monitor, set this to (640, 340) and it will st
# If you want to stream your main monitor, set this to the size of your ma
# If you want to stream a specific monitor, set this to the size of that #
```

If you are not using RGB565.

Comment out the **appropriate modules**.

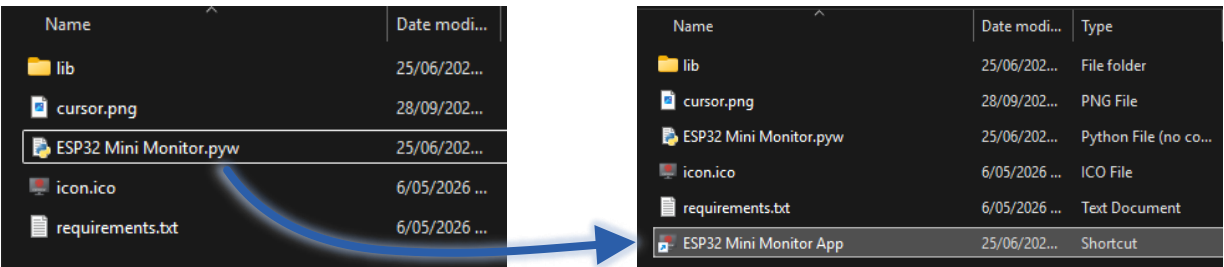
```
# Module for sending image and managing connection to esp32
# - ONLY HAVE ONE ENABLED AT A TIME
# - Uncomment the one you would like to use, and comment the other out
# - Also REQUIRES corresponding code on ESP32 side, make sure to upload the correct sketch.

from lib.esp32_tft_wifiSender_RGB565 import send_image, connect_stream    # TCP Coloured 565 or 332
#from lib.esp32_tft_wifiSender_Mono import send_image, connect_stream      # TCP Monochrome
#from lib.esp32_tft_wifiSender_Greyscale import send_image, connect_stream # TCP Greyscale
```

If you **are using** VDD (at 640 x 340 resolution) and the **RGB565 Arduino Sketch**, then **you should not have to** change anything. Main things you might want to tune are in the CONFIG section. *Everything else is somewhat commented anyway.*

Running the Python App

When you run the python app (.pyw) for the first time, it will create its own shortcut. You can then just use the shortcut.



The python app will run as a system tray icon.

More info on the last page of this doc!

*Note: If you are running in **vscode** or **another console** for testing/debugging: use the tray icon to quit, not a Keyboard Interrupt.*

ESP32 Win Monitor

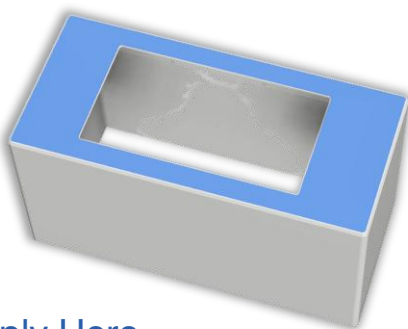
Part 2: Hardware Build Guide



*If you want to have a wooden-look front...
(or any other material/texture of your choice)*

Applying Vinyl Wrap

*If you want to have a wooden-look front...
(or any other material/texture of your choice)*



Apply Here



Cut here

This only needs to be applied to the **front surface**. There is a shroud that goes around the sides to cover up the edges (see showcase image).

It's **ok to oversize** and go a bit onto the edge surfaces though! The only important area is the screen cutout, try and get that as accurate as possible!

Do NOT cut/trim the vinyl after the screen is installed.

Otherwise, you can always just print this part in another colour!

Assembling the ESP Mount/Back Cover

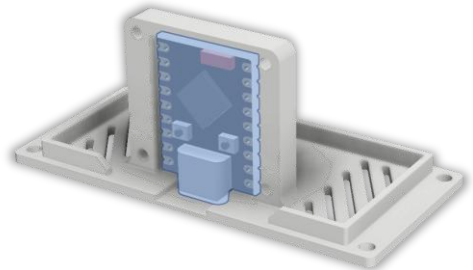
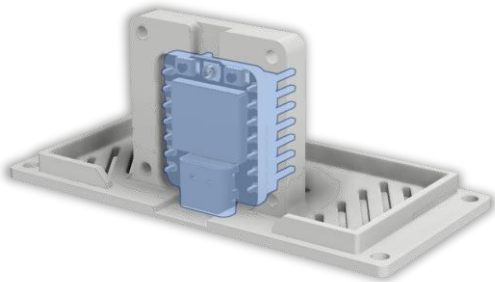
For 2 Different board variants:

Seeed Studio XIAO Boards & Generic **ESP32-S3 Supermini** §

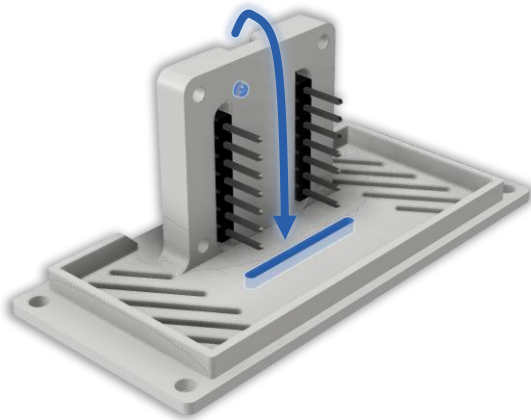
ESP32-C3 XIAO

ESP32-S3 Supermini

Place the board into the dedicated slot

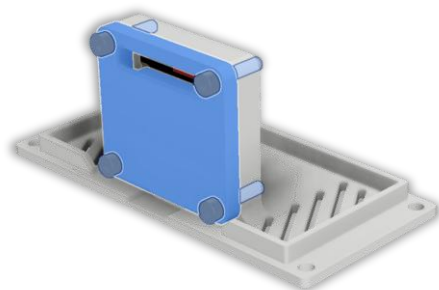
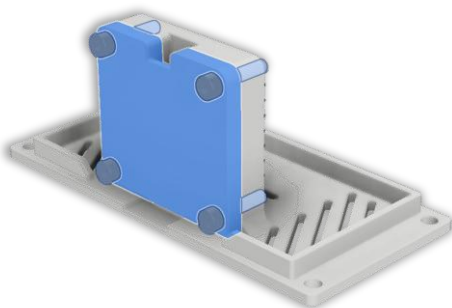


(XIAO ONLY) - Feed the Wi-Fi antenna through the back.



Not relevant
for S3
supermini.

Place the cover & fasten with M2 screws. *Length $\geq 5.00\text{mm}$*



You can get away with using only 2 screws here (same with the screen later).
Just make sure you put them in opposing corners.

Reminder: All required fasteners are on MakerWorld post.

§ The S3 supermini is cheaper (than XIAO boards) on AliExpress and is more powerful than the C3. Due to the chance that **later code releases** may be able to make use of the second core of the S3 for performance gains, I would **strongly suggest** getting an **S3 supermini** or **S3 XIAO** variant.

Installing Screen & Rear Cover

*I would recommend only doing this **AFTER** connecting all wiring / jumpers to the ESP32 after assembled in its mount.*

*Wiring diagrams are on the GitHub page. I'm not going to go into detail here - there's like 8 wires. Use female to female DuPont jumper wires, shorter ones (~10cm) better.
(About \$1 on Aliexpress)*

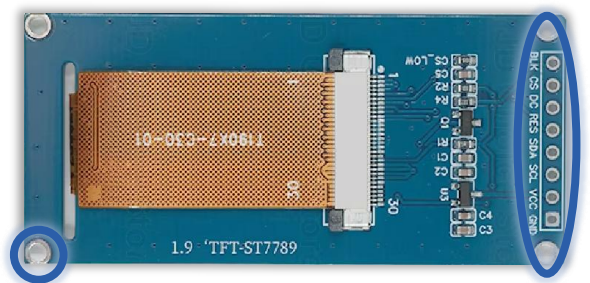
AND after applying & trimming wood vinyl (optional).

Placing Screen In Enclosure

As mentioned, complete ESP-32 Wiring First, then slide screen in:

Align the screen's headers/pins with the enclosure cutout.

Fasten with 2+ screws.



Recommended to use **M2 x 5 mm**. Use a **hex key** and **be gentle**.
A screwdriver's shaft will be too thick to reach screws.

Self-tapping screws also work. Enclosure hole is Ø1.85 mm, PCB is Ø2.00 mm.

Fastening The Enclosure

Carefully push all wires & ESP32 casing into the enclosure.
Fasten with **M2.5 x 5mm** Screws.

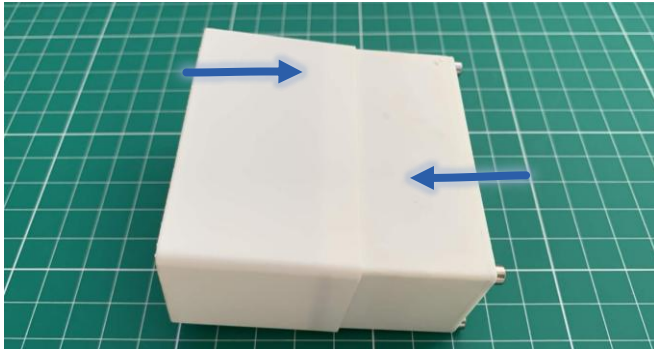


Final Assembly

Once back cover is fastened, you can add the final shroud and some adhesive rubber feet to have the screen sit at an angle.

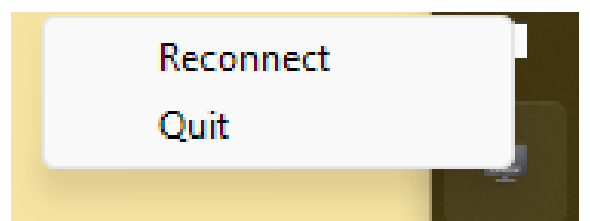
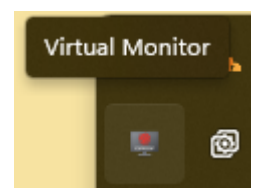
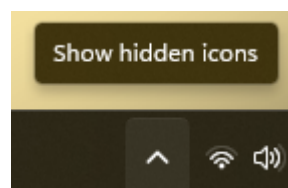
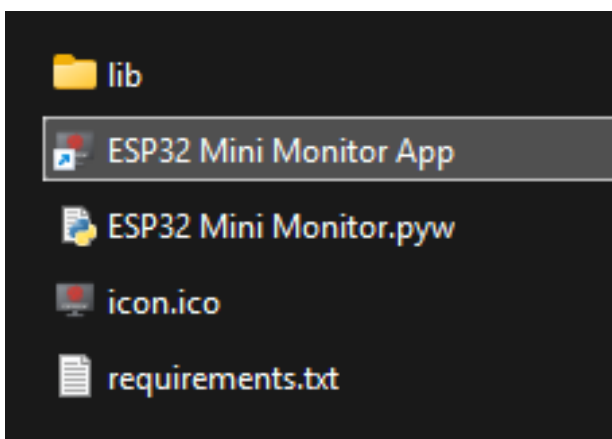
Shroud & Angled Feet

Slide the main enclosure into the shroud, optionally add feet.



Connect & You're Done!

Connect the USB-C Cable and run the python app!



Once running, right click the tray icon to force reconnect or to quit the program.

You can add the shortcut to startup apps or run it every time you turn on your PC.